

Where Did the Agent Go Wrong? Evaluating Failure Attribution Against Counterfactual Rescue

Anonymous ACL submission

Abstract

Self-improving agent loops start by attributing each failure to a step, usually by asking an LLM to read the trace. We evaluate attribution against counterfactual rescue: for every step we measure how much one correction raises the success of continued replays above uncorrected replays from that step, and score a method by this rescue gain at the step it names. On 300 natural failures from a self-improvement loop in τ^2 -bench retail and airline and in AppWorld, with registered analyses, a zero-cost rule that names the first state-changing action was competitive with every LLM method: it scored highest under the registered prompts (0.42, against 0.20–0.34), not significantly above the best, and judges asked post hoc for the quantity the metric measures tied with it. Counterfactual search did not detectably beat one judge call at seven times the cost; post hoc, giving it the judges’ grading information helped about as much as its replays did. The ranking held under a second oracle, the observed action as control, split samples and a screen of the oracle’s corrections, but not without null replays, which keep early steps from earning credit for chance recoveries. Repairs drafted from these attributions, or from the step of largest rescue gain, were practically equivalent on held-out tasks to those from the loop’s own analysis, also when the proposer could not read the failing traces and got the oracle’s diagnosis and correction instead.

1 Introduction

Agents that improve themselves from their own failures share a first step. An analyst model reads a failed episode, decides where it went wrong, and hands that diagnosis to a proposer that edits the agent’s prompts, tools or code (Shinn et al., 2023; Zhao et al., 2024; Zhang et al., 2026b; Chen et al., 2026; Xia et al., 2026). If the diagnosis points at the wrong step, the proposer may repair the wrong thing. Failure attribution has therefore become a

task of its own, with benchmarks of human-labelled multi-agent logs (Zhang et al., 2025; Liu et al., 2026), taxonomies of failure modes (Cemri et al., 2025, 2026; Zhu et al., 2025), and methods that replay a failure with a step changed (Zhang et al., 2026a; Ma et al., 2025; Lin et al., 2026; Bonagiri et al., 2026; Shah, 2026).

Three questions are rarely asked together: is an attribution correct in the sense a repair loop needs, is it worth its cost, and does a more accurate attribution make the loop’s repairs better? A human label says which step an annotator blames; a repair loop needs a step whose correction would have made the episode succeed. We evaluate replay-based methods and LLM judges against a reference defined by rescue, report cost next to the score, and hand each attribution to a working loop to measure the repairs it produces.

Defining that reference was the hard part. Naming the *earliest* step whose single correction beats replays without correction was unstable (§3), because a failure usually has several rescuing steps and which one is found first depends on the corrections an oracle happens to write. We therefore measure a *rescue profile*: for every step, the mean gain in success over null replays across four independent corrections. A method is scored by the rescue gain at the step it names, and a failure is *rescuable* when some step reaches a gain of 0.5. The null replays, which continue the unchanged episode from the same step, turned out to matter most: without them an LLM judge would rank first among the methods and the rule that wins with them sixth.

We apply the profile to 300 natural failures of agents built by a self-improvement loop, RRSI (Xia et al., 2026), in τ^2 -bench’s retail and airline domains (Barres et al., 2025) and in AppWorld with a code agent (Trivedi et al., 2024), and compare ten attribution methods, from LLM judges of different cost and design (Zhang et al., 2025) to counterfac-

tual search with up to 40 replays and two structural rules. Methods, metrics and analyses were registered before any rescue profile was computed. A second registered experiment re-runs one round of RRSI at eight of its own states in τ^2 with each attribution, or none, given to the proposer, and deploys every resulting repair on 240 held-out episodes (§6); post hoc, we repeat it with a proposer that cannot read the failing traces. Our contributions:

- **A reference by rescue profile** with null controls. It is stable across independent reruns (of 17 retested failures that both runs found rescuable, 15 had the same best rescue step), and post hoc the method ranking held across oracle samples, thresholds, a second oracle, the observed action as control, split samples and a screen for corrections that use information the agent had not seen. Without null replays the ranking reverses, because uncorrected replays from early steps often succeed by chance (§5).
- **A cost-aware comparison of ten attribution methods** on 300 natural failures. A free structural rule was competitive with all of them: the first write scored highest under the registered prompts, though not significantly above binary search (+0.08 [−0.02, +0.18]), and judges asked post hoc for the quantity the metric measures tied with it. Counterfactual search did not detectably beat a single judge call at seven times the cost; post hoc, grading information and replays each raised it by 0.06 to 0.07, to the level of the registered judges. The cheapest judge matched the most expensive, and which simple rule wins depends on the domain (§5).
- **A test on repairs.** Repairs drafted from first-write or binary-search steps, or from the step of largest rescue gain, were practically equivalent on held-out tasks to those drafted from RRSI’s own analysis (90% intervals within ± 2.5 points, though not within a typical round’s gain), and counterfactual search’s were not better. RRSI’s proposers read nearly every failing trace whatever they were shown; when they could not, the oracle’s diagnosis and correction at that step still gave no better repairs than RRSI’s analysis (§6).

2 Related Work

Failure attribution. Who&When introduced automated attribution of the responsible agent and step in multi-agent logs, with all-at-once, step-by-step and binary-search judges, and found step-level accuracy far below agent-level accuracy (Zhang et al., 2025); Who&When Pro revisits these labels (Liu et al., 2026). Taxonomies organise failures into modes, including adaptive ones (Cemri et al., 2025, 2026), and AgentDebug pairs a taxonomy with targeted feedback (Zhu et al., 2025, 2026); ErrorProbe improves diagnosis from its own history (Li et al., 2026). MP-Bench argues that a failure often admits several plausible attributions and finds that earlier conclusions about LLMs’ weakness at attribution owe much to benchmark design (In et al., 2026). These evaluations score agreement with human or injected labels; we score the effect of correcting the named step, which credits every step whose correction rescues the episode, and we likewise find that how judges are asked changes their score.

Counterfactual attribution. AgenTracer labels failures by replaying corrected steps of injected faults and trains an attributor on them (Zhang et al., 2026a); DoVer and REFLECT intervene on a trace to validate hypotheses (Ma et al., 2025; Lin et al., 2026); CausalFlow and Causal Agent Replay attribute failures by counterfactual replay (Bonagiri et al., 2026; Shah, 2026). Our reference uses the same intervention with null replays from the same step and a full profile over steps instead of the first step that flips. The profile is a graded, sampled version of but-for causation (Halpern, 2016; Pearl, 2009).

Self-improving agents. Reflexion, ExpEL and ACE turn failures into lessons or playbooks (Shinn et al., 2023; Zhao et al., 2024; Zhang et al., 2026b); HarnessFix diagnoses and repairs harness flaws (Chen et al., 2026); RRSI digests failed traces, proposes harness edits and keeps them by re-evaluation (Xia et al., 2026). Prediction of failure does not imply that intervening helps (Vasudev et al., 2026). We evaluate the diagnostic step these loops share, on failures one of them produced.

3 A Reference: Rescue Profiles

Replays. A failed episode is a sequence of agent steps $0, \dots, n-1$. A *replay from step k* keeps steps

$0, \dots, k-1$ and the environment state they produced, and lets the same agent and user simulator continue; a *corrected* replay forces a given action at step k first. The agent runs at temperature 0, but replays still vary, so each outcome is a sample.

Oracle corrections. For each step an oracle (DeepSeek-V4-Pro, thinking off) sees the domain policy, the tools, the episode up to the step and the task’s grading criteria: τ^2 ’s gold actions and required information, or AppWorld’s unit tests with their expected values. It either judges the step correct or writes the action the agent should have taken, using only what the agent knew at that point: the criteria tell it the right outcome, not identifiers or facts the agent had not yet seen.

From earliest step to profile. Our first registered protocol named the earliest step whose correction succeeded on at least 2 of 4 replays more than 4 null replays did. On 20 pilot failures, independent runs of it named the same step in 42% of pairs, mostly because the oracle corrected a step in one run and not in another. Before computing any profile on the main set, we registered protocol 3 instead.

Protocol 3. At every step k we draw $K = 4$ oracle samples. Each sample that corrects the step gets 2 corrected replays, and the step gets 4 null replays. The rescue gain of step k is

$$R_k = \frac{1}{K} \sum_{i=1}^K c_{k,i} (\hat{p}_{k,i}^{\text{corr}} - \hat{p}_k^{\text{null}}), \quad (1)$$

where $c_{k,i} = 1$ if sample i gives an action the replay can apply, and $c_{k,i} = 0$ if it judges the step correct or its action cannot be applied. R_k is thus the share of samples that propose a correction times the mean gain of those corrections. A null replay lets the agent act again at step k . It reproduced the observed tool calls in 91–95% of τ^2 null replays and the observed code in 58% of AppWorld ones (post hoc), so R_k compares forcing the oracle’s action with letting the agent act again, which in τ^2 is close to keeping the observed action. A failure is rescuable if $\max_k R_k \geq 0.5$, and then its best rescue step is the step with the largest gain (earliest on ties). We also record the earliest step with $R_k \geq 0.5$. Figure 1 shows one profile. On the pilot, two independent runs agreed on the best rescue step for 15 of 20 failures.

Scoring a method. A method names one step $\hat{k}$. Its score on a failure is $R(\hat{k})$, the rescue gain at

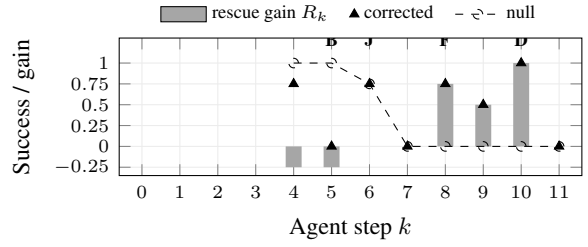


Figure 1: Rescue profile of one retail failure. Corrected: mean success of the oracle’s corrections; null: success of uncorrected replays (run only where some sample proposed a correction). Steps 8–10 all rescue the episode; D marks the best rescue step, F the first write, J the all-at-once judges’ step and B binary search’s. At step 6, three of four corrections succeeded, but so did three of four uncorrected replays.

that step, and 0 for an invalid step. The primary metric is the mean of $R(\hat{k})$ over rescuable failures; it rewards naming any step whose correction helps, in proportion to how much it helps. Secondary metrics include exact agreement with the best rescue step (Appendix Table 4).

4 Setup

Failures. All failures come from RRSI (Xia et al., 2026) run as a self-improvement loop on a DeepSeek-V4-Flash agent: from the harnesses it accepted over 20 rounds, in their evaluations on training tasks and in their deployment on held-out tasks. In τ^2 -bench (Barres et al., 2025) the agent serves a simulated customer through tool calls; in AppWorld (Trivedi et al., 2024) it writes Python against the APIs of nine apps and submits with a final call. We drew 100 failed episodes per domain with a registered rule: no harness error, at most two per task and harness, tasks taken round-robin in a seeded order. They cover 36 (retail), 24 (airline) and 47 (AppWorld) tasks and average 11.7, 10.3 and 19.6 agent steps.

Methods. Every LLM method except counterfactual search reads the failure as RRSI’s analyst reads it: the task (in τ^2 with the simulated customer’s hidden instructions), the episode with numbered steps, and the grading report the agent never sees, which lists τ^2 ’s gold actions and missing information or AppWorld’s failed unit tests with their assertions. The judges therefore have the information the oracle has. Each method names one step.

- **Rules:** the *first write*, the first step that calls a state-changing tool or API (else the last step),

and the *last step*.

- **All-at-once** judge (Zhang et al., 2025): one call over the whole episode, with DeepSeek-V4-Flash, V4-Pro, and V4-Pro with thinking.
- **Step-by-step** judge: one call per step, in order, until a step is judged decisive.
- **Binary search** judge: halves the episode $\lceil \log_2 n \rceil$ times, asking which half holds the decisive mistake.
- **Root-cause attributor**: one call asking for a root cause, the earliest wrong step and a component.
- **RRSI trace digester** (Xia et al., 2026): RRSI’s own multi-turn failure digest; we take the earliest step it cites as evidence.
- **Counterfactual search** (CF search), replay-based like AgenTracer’s labelling (Zhang et al., 2026a) but without grading criteria: one call names up to five suspect steps with corrected actions; suspects are tested in order with 4 corrected and 4 null replays, and the search stops at the first correction that beats null by 2 successes or when its budget of 40 replays runs out; otherwise it answers with the first suspect.

Judges are prompted for the decisive mistake, defined as the earliest step whose correction would most likely have rescued the episode (Appendix B); post hoc reruns change this definition or withhold the grading information. Pro runs with thinking off unless stated.

Registration and analysis. Methods, prompts, metrics and analyses were registered, and all methods had been run, before the main profiles were computed; no method sees a profile. The registered primary comparison is counterfactual search at budget 40 against the all-at-once judge with Pro: the paired difference in $R(\hat{k})$ over rescuable failures, with a 95% bootstrap interval over task clusters (10,000 draws) and a paired permutation test flipping signs by task, on retail and airline pooled and, for AppWorld, separately. Everything else is estimated with cluster-bootstrap intervals and no further tests; unregistered analyses are marked post hoc or exploratory. A seeded 10% of failures in each domain got a second, independent profile. Costs are list-price dollars for tokens and replays. Appendix F lists every change made after registration.

5 Results

Profiles. Of 300 failures, 157 were rescuable: 65 of 100 in retail, 37 in airline and 55 in AppWorld. Null replays from the same step succeeded 27%, 18% and 11% of the time; without them, many rescues would be chance recoveries. A third of the rescuable failures (18, 11 and 26) had two or more steps with a gain of at least 0.5. In the retest, 27 of 30 failures got the same verdict on rescuability, 15 of the 17 that both runs found rescuable the same best rescue step (the other two one and two steps apart), and step-level gains correlated .63 to .80.

Under the registered prompts, the first-write rule scored highest. Table 1 gives every method’s rescue gain. Pooled over the three domains, the first write scored 0.42 [0.33, 0.51]; the LLM methods scored between 0.20 (step-by-step) and 0.34 (binary search). The first write’s lead over binary search was not significant (+0.08, exploratory, $[-0.02, +0.18]$); over the all-at-once judges it was +0.11 to +0.12. Exact agreement with the best rescue step ordered the methods the same way (Appendix Table 4). At the steps the methods named, the oracle proposed a correction about equally often (0.52–0.64 of samples); the first write led because corrections at its steps worked better (mean gain 0.61, against 0.34–0.51; post hoc, Appendix Table 9). Cost bought little: the all-at-once judges with Flash (\$0.001 per failure), Pro (\$0.003) and Pro with thinking (\$0.014) scored 0.30, 0.30 and 0.31 (Appendix Figure 3). On Who&When our judges reproduce the published pattern (Appendix D).

Replays and grading information both helped search. Counterfactual search at budget 40 scored 0.26 pooled. The registered comparison with the all-at-once judge (Pro) detected no difference, with intervals that admit 0.1 either way: $-0.061 [-0.146, +0.032]$ in τ^2 (permutation $p = .20$, $n = 102$) and $+0.025 [-0.087, +0.136]$ in AppWorld ($p = .69$, $n = 55$). It confounds replays with information, since search does not read the grading criteria. Post hoc, we reran search’s suspect call with the grading information the judges read, keeping its replays and budget (\$2.4), which completes a 2×2 of replays and information (Table 1, lower block). Search’s first suspect scored 0.22 without the information and 0.27 with it; after replays, 0.26 and 0.35. Information and replays raised the score by similar amounts (main effects

Method	Model	Mean rescue gain $R(\hat{k})$, rescuable failures [95%]				Exact (pooled)	\$ per failure
		Retail (65)	Airline (37)	AppWorld (55)	Pooled (157)		
First write	rule	0.53 [0.38, 0.65]	0.27 [0.13, 0.45]	0.39 [0.26, 0.52]	0.42 [0.33, 0.51]	0.38	0.0000
Last step	rule	0.11 [0.02, 0.24]	0.13 [0.04, 0.23]	0.36 [0.23, 0.50]	0.20 [0.13, 0.29]	0.13	0.0000
All-at-once	Flash	0.29 [0.19, 0.38]	0.41 [0.28, 0.53]	0.25 [0.14, 0.37]	0.30 [0.24, 0.37]	0.28	0.0010
All-at-once	Pro	0.30 [0.19, 0.41]	0.34 [0.21, 0.48]	0.26 [0.15, 0.38]	0.30 [0.23, 0.36]	0.31	0.0031
All-at-once	Pro, thinking	0.26 [0.17, 0.35]	0.42 [0.28, 0.57]	0.28 [0.18, 0.40]	0.31 [0.24, 0.38]	0.32	0.0141
Step-by-step	Pro	0.14 [0.03, 0.25]	0.16 [0.04, 0.25]	0.29 [0.18, 0.40]	0.20 [0.13, 0.26]	0.19	0.0099
Binary search	Pro	0.33 [0.21, 0.47]	0.34 [0.21, 0.46]	0.35 [0.24, 0.45]	0.34 [0.27, 0.41]	0.32	0.0064
Root-cause attributor	Pro	0.26 [0.16, 0.38]	0.36 [0.21, 0.53]	0.22 [0.10, 0.34]	0.27 [0.20, 0.34]	0.28	0.0031
RRSI trace digester	Pro	0.17 [0.07, 0.29]	0.35 [0.24, 0.49]	0.19 [0.11, 0.27]	0.22 [0.16, 0.29]	0.21	0.0077
Counterfactual search@40	Pro, replays	0.22 [0.12, 0.33]	0.31 [0.19, 0.44]	0.28 [0.17, 0.41]	0.26 [0.20, 0.34]	0.26	0.0222
<i>Post hoc variants</i>							
All-at-once, gain	Pro	0.39 [0.28, 0.51]	0.42 [0.28, 0.55]	0.29 [0.18, 0.42]	0.36 [0.29, 0.44]	0.36	0.0031
Binary search, gain	Pro	0.44 [0.33, 0.55]	0.42 [0.25, 0.60]	0.35 [0.23, 0.47]	0.40 [0.33, 0.48]	0.39	0.0071
All-at-once, no grading	Pro	0.17 [0.08, 0.26]	0.32 [0.19, 0.45]	0.25 [0.14, 0.35]	0.23 [0.17, 0.30]	0.24	0.0028
Binary search, no grading	Pro	0.21 [0.10, 0.33]	0.24 [0.12, 0.35]	0.24 [0.14, 0.35]	0.23 [0.16, 0.29]	0.20	0.0064
CF search@0	Pro	0.22 [0.14, 0.32]	0.13 [0.02, 0.24]	0.28 [0.17, 0.40]	0.22 [0.16, 0.28]	0.20	0.0037
CF search@0, grading	Pro	0.26 [0.17, 0.37]	0.24 [0.13, 0.34]	0.31 [0.18, 0.44]	0.27 [0.21, 0.34]	0.27	0.0037
CF search@40, grading	Pro, replays	0.34 [0.25, 0.45]	0.38 [0.26, 0.52]	0.34 [0.22, 0.46]	0.35 [0.28, 0.42]	0.33	0.0152

Table 1: Attribution scored by the rescue gain at the named step. Intervals: cluster bootstrap over tasks. Exact: share naming the best rescue step. Cost: mean over domains of list-price dollars per failure (tokens and replays). Lower block, post hoc (§5): judges asked for the largest gain over null (gain) or without grading information; search’s first suspect before any replay (@0) and search given the grading information.

+0.07 [+0.01, +0.13] and +0.06 [+0.03, +0.10]; replays +0.08 in τ^2 , +0.02 in AppWorld), with no clear interaction (+0.03 [−0.02, +0.09]). Informed search confirmed a suspect on 76–80% of failures, against 42–73%, with fewer replays. It matched the registered binary-search judge (+0.01 [−0.06, +0.08]) and fell short of the first write (−0.07 [−0.18, +0.03]) and of binary search asked for the largest gain (−0.06 [−0.12, +0.01]), at \$0.015 per failure against \$0.006. Withholding the information from the judges cost them about as much: 0.07 [0.00, 0.13] for the all-at-once judge and 0.11 [0.02, 0.20] for binary search. These contrasts are post hoc and descriptive, and our search is not a reimplement of AgenTracer, which trains an attributor on replay labels. Registered search stopped improving at 24 replays (Appendix C).

Asked for the measured quantity, judges tied with the rule. Judges that missed the best rescue step usually named an earlier one (in retail, the all-at-once judge with Pro on 40 of 65 rescuable failures). The registered prompts asked for the earliest decisive mistake, while R_k measures the gain of a correction over letting the agent act again. Post hoc, we re-ran three judges asking instead for the step whose correction would most likely rescue the task, or would most increase success over let-

ting the agent act again (Appendix B). The second, which matches the metric, raised the Pro judges by 0.07 (all-at-once 0.30 to 0.36, [+0.02, +0.12]; binary search 0.34 to 0.40, [−0.01, +0.13]; Table 1) and the Flash judge by 0.01. Binary search asked this way tied with the first write (−0.02 [−0.10, +0.07]), so the first write’s lead under the registered prompts owes much to how the judges were asked.

The best rule depends on the domain. The first write led in retail (0.53, against 0.14–0.33 for LLM methods) and AppWorld (0.39); in airline the all-at-once judges led (0.41–0.42, against 0.27). Most best rescue steps were writes in retail (36 of 65), usually a wrong write replaced by the right one, and utterances in airline (25 of 37), typically a refusal or a clarifying question the agent should have made before acting. In AppWorld they lay late, on average 77% of the way through the episode, so the last step scored 0.36 (Appendix Table 3). Binary search was the only LLM method whose score held across domains (0.33–0.35). A rule that reads τ^2 ’s gold actions, as the judges do, scored 0.50 (post hoc; Appendix Table 8), though the oracle corrects from the same gold actions.

The ranking is robust, except to the null control. The reference rests on choices: four oracle

samples per step, a threshold of 0.5, null replays as the control, one oracle model, and an oracle that reads the grading. Appendix Table 6 re-scores every method under other choices (post hoc; details in Appendix D). With fewer samples, thresholds from 0.25 to 0.75, only failures with a single rescuing step, or failures chosen on two samples and scored on the other two, the first write stayed first and binary search the best LLM method (Kendall’s τ of 0.78 to 1.00). With the observed action forced as the control, the judges gained most, and binary search asked for the largest gain tied with the first write (0.45 against 0.44, -0.01 $[-0.10, +0.07]$). A second oracle model (V4-Flash) found 120 failures rescuable instead of 157 and agreed with the first on the best rescue step for 71–72% of those both found rescuable, less often than two runs of the same oracle did; under its profile the first write still ranked first ($\tau = .78$). The oracle may use the grading only to know the right outcome. A mechanical screen of the 4,795 corrections that entered a profile (Appendix Table 11) found an identifier, name or date the agent had not yet seen in 0.7% and two or more calls beyond the observed step in 1.8%; dropping them left the ranking unchanged ($\tau = 1.00$). Dropping the null replays reversed it: 203 failures became rescuable, the first write fell to sixth place, and the all-at-once judge with thinking ranked first (0.42 against 0.37). Uncorrected replays from the first fifth of an episode succeeded 41–44% of the time and from the last fifth 2–6% (Appendix Figure 2), and LLM methods other than binary search named steps on average 53–67% of the way through an episode, against 77% for best rescue steps, so without the null control they collect credit for recoveries that happen anyway. Replay-based labels (Zhang et al., 2026a; Ma et al., 2025; Bonagiri et al., 2026; Shah, 2026) need such a control.

6 Do Better Attributions Give Better Repairs?

Design. A rescue score is a proxy: a loop needs attributions only to repair its agent. We tested this on RRSI itself, in a separately registered experiment. We took two other runs of the loop in each τ^2 domain and the harness each run had accepted by the start of rounds 5 and 15, giving eight incumbent states. At each state we re-ran one unmodified RRSI round five times, changing only the evidence the proposer gets:

- **No attribution:** a note asking the proposer to read the traces itself.
- **RRSI analysis:** the round’s own analysis report and trace digests, unchanged.
- **First write, binary search (Pro) and counterfactual search@40:** for each failing trace, the step the method named and an excerpt of it, with no reason, component or method name (Appendix B).
- **Oracle step** (post hoc, after the registered analysis): the same digest naming the step with the largest rescue gain in the trace’s own profile, for the 60 traces where some step had a positive gain.
- **No reading** (post hoc, after the third review): two groups whose proposer cannot open failing traces (a read returns a notice; passing traces stay readable), given either RRSI’s analysis, or for the same 60 traces the oracle step with its excerpt, the oracle’s diagnosis and the correction that rescued most often there (*oracle fix*).

The proposer works from the traces RRSI builds from the incumbent’s evaluation on training tasks (81 failing traces in all, 42 retail and 39 airline) and, as in RRSI, can read each of them, grading report included, except in the no-reading groups. Each round drafts two candidates and puts them through RRSI’s critic, smoke test and full evaluation on training tasks, after which RRSI’s own rule decides whether to accept one. A candidate is *deployable* if it passes these gates with a harness error rate of at most 2%. Every deployable candidate and every incumbent was deployed on 240 held-out episodes (retail 40 tasks $\times$ 6, airline 20 $\times$ 12).

Analysis. A candidate slot’s gain G is its held-out success minus its incumbent’s, in points, and 0 if it was not deployable, since the loop then keeps the incumbent. A contrast between two groups is the mean over the eight states of the difference in their mean gains. The six primary contrasts compare each step attribution with no attribution and with RRSI’s analysis; p values come from permuting group labels within states and are Holm-adjusted, and 95% intervals from a t distribution over states (df 7). A contrast whose 90% interval lies within ± 2.5 points, about one standard deviation of a single candidate’s gain in earlier runs, counts as practically equivalent. The margin reflects the noise of

Evidence for the proposer	$R(\hat{k})$, these		Mean G , all 16 slots			Deployable		RRSI	Round
	retail / airline	Deployable	both	retail	airline	mean G	$G > 0$	accepted	level
No attribution	–	10/16	−4.38	−9.27	+0.52	−7.00	50%	2	−0.10
RRSI analysis	0.30 / 0.21	13/16	−0.34	−0.31	−0.36	−0.42	38%	1	−0.10
First write	0.37 / 0.16	10/16	−0.03	+0.21	−0.26	−0.04	60%	2	+0.47
Binary search	0.32 / 0.51	9/16	−0.05	+0.16	−0.26	−0.09	44%	0	0.00
Counterfactual search@40	0.27 / 0.32	12/16	−2.47	−0.26	−4.69	−3.30	25%	2	+0.16
Oracle step	0.67 / 0.81	10/16	−0.39	−1.04	+0.26	−0.62	50%	1	+0.26
RRSI analysis, no reading	0.30 / 0.21	14/16	−0.39	−0.42	−0.36	−0.45	21%	2	−0.05
Oracle fix, no reading	0.67 / 0.81	10/16	0.00	+0.42	−0.42	0.00	60%	1	+0.10

Primary contrast	Difference [95%]	p	Holm	Equivalent	Round level [95%]
First write – none	+4.35 [−5.97, +14.67]	0.52	1.00	no	+0.57 [−0.19, +1.34]
Binary search – none	+4.32 [−6.90, +15.55]	0.66	1.00	no	+0.10 [−0.14, +0.35]
CF search@40 – none	+1.90 [−10.24, +14.04]	0.91	1.00	no	+0.26 [−0.99, +1.51]
First write – RRSI	+0.31 [−0.56, +1.18]	0.65	1.00	yes	+0.57 [−0.32, +1.46]
Binary search – RRSI	+0.29 [−0.82, +1.39]	0.60	1.00	yes	+0.10 [−0.14, +0.35]
CF search@40 – RRSI	−2.14 [−5.67, +1.40]	0.31	1.00	no	+0.26 [−0.99, +1.51]
Oracle – RRSI	−0.05 [−1.97, +1.87]	0.98	–	yes	+0.36 [−0.27, +0.99]
Oracle – step groups	+0.46 [−1.62, +2.54]	0.75	–	yes	–
Oracle fix – RRSI, no reading	+0.39 [−0.98, +1.76]	0.53	–	yes	+0.16 [−1.44, +1.76]
RRSI: no reading – reading	−0.05 [−1.31, +1.21]	0.97	–	yes	+0.05 [−1.55, +1.66]
Oracle fix, no reading – oracle step	+0.39 [−0.78, +1.56]	0.61	–	yes	−0.16 [−0.85, +0.54]

Table 2: Repairs drafted from each kind of evidence in τ^2 : 8 states $\times$ 2 candidates per group. G : held-out success of a candidate minus its incumbent’s, in points, 0 if not deployable. $R(\hat{k})$: rescue gain of the steps a group pointed to, on the 33 rescuable failures among the experiment’s 81 failing traces (post hoc; for RRSI, the earliest step its digests cite). Accepted: by RRSI’s own rule. Round level: gain of the accepted candidate, 0 if none (tested by exact sign flips, text). Contrasts: mean over states of group differences, p from within-state permutation; Equivalent: 90% interval within ± 2.5 points. The oracle and no-reading groups and their contrasts are post hoc and not Holm-adjusted.

one deployment, not a threshold of practical value: an accepted round of the loop rarely gains more than a point, so we also report the smallest margin each interval excludes. The interval treats the eight states as independent, though they come from four runs of the loop, two checkpoints each; grouping them by run is a post hoc check. The design, with an expected standard error of 0.9 points per contrast, was registered before any attribution was run on these traces, and no held-out result was looked at until all 62 deployments had finished.

No contrast was significant. Of 80 drafts, 54 were deployable, 9 to 13 per group (Table 2); none failed the error check. No primary contrast was significant (Holm-adjusted $p = 1.0$ for all six). Repairs drafted from first-write and binary-search steps were practically equivalent to those drafted from RRSI’s analysis: +0.31 [−0.56, +1.18] and +0.29 [−0.82, +1.39] points; their 90% intervals exclude differences beyond 1.0 and 1.2 points, or 1.5 and 1.4 points with the states grouped by run, but not differences the size of a typical round’s

gain. Counterfactual search was 2.14 points below RRSI’s analysis [−5.67, +1.40], mostly through two airline candidates that lost 23 and 14 points. The contrasts with no attribution span about ± 10 points because of one candidate drafted without attribution, whose write gate silently blocked nearly every write (−72.5 points); without it, that group’s mean gain was +0.17 (exploratory).

The step groups differed little in rescue score.

Post hoc, we computed rescue profiles for the 81 failing traces these proposers saw (\$5.0); only 33 were rescuable (17 retail, 16 airline). On them the first write scored 0.27, binary search 0.41, counterfactual search 0.30 and the earliest step RRSI’s own digests cite 0.26 (Table 2): the first write was no better a pointer here than RRSI’s analysis (+0.01 [−0.15, +0.19]), and binary search better but not significantly (+0.15 [−0.03, +0.35]). Across the 64 candidate slots of the step and oracle groups, the rescue score of the pointers a proposer received correlated only weakly with its candidates’ gains (Spearman .15 within states, permutation $p = .14$).

Most repairs did not help on held-out tasks in any group: 21–60% of deployable candidates gained, and no group’s mean gain was above zero. RRSI’s own rule rejected all three candidates that lost more than 10 points, so the gain of the accepted candidate differed little between groups (-0.10 to $+0.47$ points; exact sign-flip $p \geq .25$).

An oracle pointer did not help either. To test a pointer that scores highest by construction, we ran a sixth group post hoc with the step of largest rescue gain (0.67 and 0.81 on rescuable failures in retail and airline; \$5.9), keeping its held-out results unseen until its 10 deployments had finished. Its candidates’ mean gain was -0.39 points, practically equivalent to RRSI’s analysis (-0.05 [$-1.97, +1.87$]; within ± 2.1 with states grouped by run) and to the three registered step groups pooled ($+0.46$ [$-1.62, +2.54$], though not once grouped by run).

Proposers read the traces themselves. The proposers’ action logs suggest why the evidence mattered so little (post hoc). In every group the proposer opened 97–100% of the failing traces, 13 to 20 reads per draft, mostly of the whole trace or a long window, and without any attribution 95–97% of the reads of a failing trace covered the step each of the four pointers named for it.

Without reading, an oracle’s diagnosis and correction did not help either. To give proposers evidence they could not find themselves, we ran the two no-reading groups post hoc (\$12.4), keeping their held-out results unseen until all their deployments had finished. Their proposers tried 1.5 to 2 blocked reads per draft and read 2.3 to 3.5 passing traces instead. Blocking the reads did not change the repairs drafted from RRSI’s analysis (-0.05 [$-1.31, +1.21$] points against the same evidence with reading), and the oracle’s step, diagnosis and correction did not improve on RRSI’s analysis ($+0.39$ [$-0.98, +1.76$]; 90% interval within ± 1.5 , or ± 2.0 with states grouped by run). From the oracle’s evidence 6 of 10 deployable candidates gained, against 3 of 14, but their mean gain was 0.00 points, and accepted candidates gained $+0.10$ against -0.05 (sign-flip $p = .75$).

7 Discussion

What the score measures. R_k belongs to a step and an oracle: how often the oracle proposes a correction, times how well its corrections work. Cor-

rections of writes worked better than corrections of utterances (in retail 0.52 against 0.13; Appendix Table 10), since knowing the expected outcome suffices to write the right call, and a second oracle agreed with the first less than the first agreed with itself. A low R_k can therefore mean that this oracle could not repair a step, not that the step did not matter, and a high R_k marks an effective point of recovery, which need not be where the failure began. The profile is a reference for intervention, not an independent record of where the error lies; the ranking of methods, however, held across both oracles, both controls and the screen of corrections.

Why a free rule is competitive. In customer service a wrong database write often decides a failure, and the first write is often that write; where the decisive mistake was conversational, as in airline, judges led. A judge’s score needs a domain baseline and a definition of the target that matches its use.

Implications for self-improving loops. Neither a pointer to the step of largest rescue gain, with about three times the rescue score of RRSI’s own analysis on the same failures, nor that step’s diagnosis and correction given to a proposer that could not read the traces, gave repairs better than the loop’s own analysis (§6). This suggests that in this loop the difficulty lay after attribution, in turning a diagnosis into a harness edit that helps on unseen tasks: most edits in every group moved held-out success by less than 5 points (Appendix Figure 4). RRSI’s re-evaluation rejected every edit that misfired broadly: the evaluation that decides which repairs to keep protected the agent more than the attribution behind them.

8 Conclusion

Against counterfactual rescue with null controls, a free structural rule was competitive with LLM attribution, and grading information helped counterfactual search as much as its replays did. Neither the step of largest rescue gain nor its diagnosis and correction improved the repairs of the loop we studied, whether or not its proposer could read the failing traces. Rescue profiles rank methods consistently across retests, oracles, controls and screens of the corrections, but only with null replays. We release failures, profiles and code.

Limitations

All agents, oracles and attributors use one model family (DeepSeek-V4); the second oracle is a smaller model of the same family. The failures come from one self-improvement loop and three domains, with 37 to 65 rescuable failures each, so per-domain intervals are wide. The reference rests on one kind of intervention, a single action corrected by an oracle that sees the grading: it says where a correction helps, not where an error began, and the oracle repaired writes more readily than utterances. A mechanical screen found few corrections that use information the agent had not seen or do more than one step, but no human has audited the profiles; we release a stratified sample of 60 corrections with an audit sheet for this purpose. The threshold of 0.5 and four samples per step were fixed on a 20-failure pilot. Many analyses are post hoc: the robustness checks, the observed-action control, the second oracle, the admissibility screen, the prompt variants, the judges without grading information, the informed search, the gold-deviation rule, the reading logs, and in the repair experiment the rescue scores of its pointers, its run-grouped intervals, and its oracle and no-reading groups. Our registered prompts asked for the earliest decisive mistake while the metric rewards the largest gain over null, so the registered comparison understates the judges. Our counterfactual search labels steps by replay but trains no attributor, unlike AgenTracer (Zhang et al., 2026a), so its results do not speak to trained attributors. Fourteen retail failures were drawn after the others, from the two further runs of the loop that the repair experiment also used (one trace is in both). Two of 6,540 AppWorld replays could not run (Appendix F). Who&When is a multi-agent benchmark with a different format; only methods without replay were run there. The repair experiment covers τ^2 only, with eight states from four runs of the loop, 16 candidate slots per group and 33 rescuable failures among its 81 failing traces. Its margin of 2.5 points reflects the noise of one deployment and exceeds a typical round’s gain, and its intervals widen when states are grouped by run; it rules out large differences from RRSI’s analysis, not small ones, and its contrasts with no attribution are dominated by one candidate. The registered digests carried only the step; the post hoc groups added a diagnosis and a correction, but no group named a harness component to change, and all used one loop and

one proposer model.

Ethics Statement

All environments use synthetic users and data. Attribution methods that misdiagnose failures can lead self-improving agents to change the wrong behaviour; our results suggest that such diagnoses should be checked against simple baselines before they drive automatic changes.

References

- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.
- Akash Bonagiri, Devang Borkar, Gerard Janno Anderias, Setareh Rafatirad, and Houman Homayoun. 2026. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures. *arXiv preprint arXiv:2605.25338*.
- Mert Cemri, Andrei Cojocaru, Melissa Pan, Shu Liu, Shubham Agarwal, Alexander Krentsel, Jay Tang, Kannan Ramchandran, Joseph E. Gonzalez, Matei Zaharia, Alexandros G. Dimakis, and Ion Stoica. 2026. Fantastic adaptive taxonomies and how to use them. *arXiv preprint arXiv:2607.16387*.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*.
- Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. 2026. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*.
- Joseph Y. Halpern. 2016. *Actual Causality*. MIT Press.
- Yeonjun In et al. 2026. Rethinking failure attribution in multi-agent systems: A multi-perspective benchmark and evaluation. In *International Conference on Machine Learning*. ArXiv:2603.25001.
- Jiazheng Li, Emine Yilmaz, Bei Chen, and Dieu-Thu Le. 2026. Towards self-improving error diagnosis in multi-agent systems. *arXiv preprint arXiv:2604.17658*.
- Xiaofeng Lin, Yingxu Wang, Tung Sum Thomas Kwok, Daniel Guo, Sahil Arun Nale, Charles Fleming, and Guang Cheng. 2026. REFLECT: Intervention-supported error attribution for silent failures in LLM agent traces. *arXiv preprint arXiv:2606.09071*.

Jiale Liu, Huajun Xi, Shaokun Zhang, Yifan Zeng, Tianwei Yue, Chi Wang, Jian Kang, Qingyun Wu, and Huazheng Wang. 2026. Who&when pro: Can LLMs really attribute failures in AI agents? *arXiv preprint arXiv:2607.09996*.

Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2025. DoVer: Intervention-driven auto debugging for LLM multi-agent systems. *arXiv preprint arXiv:2512.06749*.

Judea Pearl. 2009. *Causality: Models, Reasoning, and Inference*, 2nd edition. Cambridge University Press.

Jaineet Shah. 2026. Causal agent replay: Counterfactual attribution for LLM-agent failures. *arXiv preprint arXiv:2606.08275*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjana Balasubramanian. 2024. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

Rakshith Vasudev, Melisa Russak, Dan Bikel, and Waseem Alshikh. 2026. Accurate failure prediction in agents does not imply effective failure prevention. *arXiv preprint arXiv:2602.03338*.

Peng Xia, Rujun Han, Zifeng Wang, Yanfei Chen, Yufan Zhuang, Yoonho Lee, Chengsong Huang, Han Yu, Zhongying CuiZhu, Yifei Ming, Huaxiu Yao, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. 2026. RRSI: Regularized recursive self-improvement of agent harnesses. *arXiv preprint arXiv:2609.24972*.

Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. 2026a. AgentTracer: Who is inducing failure in the LLM agentic systems? In *International Conference on Learning Representations*. ArXiv:2509.03312.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026b. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*. ArXiv:2505.00212.

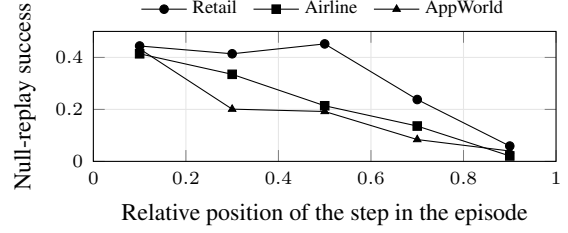


Figure 2: Success of uncorrected (null) replays from a step, by the step’s position in the failed episode (fifths), at the steps where the oracle proposed a correction. Each point averages 27 to 35 steps in the first fifth and 81 to 323 in the others.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. 2025. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.

Kunlun Zhu, Xuyan Ye, Zhiguang Han, Yuchen Zhao, Bingxuan Li, Weijia Zhang, Muxin Tian, Xiangru Tang, Pan Lu, James Zou, Jiaxuan You, and Heng Ji. 2026. AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents. *arXiv preprint arXiv:2607.18754*.

A Profile Details

Replays. τ^2 replays restore the database state after the kept steps and continue with the same agent harness and user simulator (DeepSeek-V4-Flash, temperature 0); a corrected action replaces the agent’s turn at step k (a message, or tool calls whose results are then computed). AppWorld replays restore the app state by re-executing the kept code and force the corrected turn, which must contain one Python block; the episode continues with the incumbent harness. A correction the replay cannot apply counts as giving no gain.

Retest. The retest failures were drawn with a seeded random choice of 10% per domain before any profile was computed and run with fresh oracle samples and replays.

Cost. The rescue profiles cost \$11.1 for the first 186 τ^2 failures and \$13.7 for AppWorld (plus \$0.7 for its retest); all parts of the 14 τ^2 failures added later cost about \$1.5. The attribution study, including methods, search, pilots and Who&When, cost

about \$58, and the post hoc analyses for the main set \$23.1: judges with other definitions or without grading information \$9.4, the observed-action control \$3.9 and the second oracle \$9.8. The repair experiment (§6) cost about \$35: \$6.4 for drafting and critique, \$1.8 for attribution and search replays, \$7.9 for RRSI’s evaluations and smoke tests (5,639 episodes) and \$18.9 for held-out deployment (14,880 episodes); post hoc, its failures’ rescue profiles cost \$5.0 and the oracle-pointer group \$5.9.

B Prompts

The τ^2 prompts follow, verbatim; braces mark what is filled in. The AppWorld prompts replace the description of the agent and the task, the grading report and the component descriptions with AppWorld’s (the code agent, its unit tests, and the same nine components worded for it), and the oracle and search ask for one assistant turn with one Python block.

Shared introduction (all judges)

A customer-service agent (an LLM inside a harness) failed a
 ↳ task of tau2-bench. Below is the failure as the developers’
 ↳ analysis tools see it: the customer’s hidden instructions,
 ↳ the conversation (each agent step is numbered), and the
 ↳ grading.

The DECISIVE MISTAKE is the earliest agent step such that, had
 ↳ the agent acted correctly at that step and continued
 ↳ normally, the task would most likely have succeeded.

Harness components (a fix to the harness changes one of these):
 - prompt: the system prompt text (instructions and policy
 ↳ wording the model reads)
 - control_flow: the agent loop code: how the next action is
 ↳ produced, checks or gates applied to tool calls before they
 ↳ run, retries, enforced confirmations
 - config: numeric settings such as temperature, token limits or
 ↳ step limits
 - output_plumbing: how tool results and tool descriptions are
 ↳ rendered to the model
 - context_mgmt: which parts of the conversation history the
 ↳ model sees
 - client_tool: extra tools the harness gives the model (e.g. a
 ↳ checker or calculator)
 - skill: instruction files loaded for specific situations
 - memory: structured state the harness keeps across steps
 - subagent: an extra model call, e.g. a second model reviewing
 ↳ an action

```
=== FAILURE ===
{the failure: hidden instructions, numbered steps, grading
↳ report}
```

All-at-once

Find the decisive mistake and the harness component whose
 ↳ change would most likely have prevented it. Answer with
 ↳ JSON:
 {"step": <step number>, "top3": [<the three most likely step
 ↳ numbers, best first>],
 "component": "<component>", "components3": [<"three most
 ↳ likely components, best first>"],
 "reason": "<one sentence>"}

Step-by-step (asked for $k = 0, 1, \dots$ on the episode up to step k)

Judge step {k} only, given what happened before it. Is the
 ↳ agent’s action at step {k} the decisive mistake (the
 ↳ earliest step whose correction would most likely have made
 ↳ the task succeed)? Answer with JSON:
 {"decisive": true | false, "component": "<component, if
 ↳ decisive>", "reason": "<one sentence>"}

Binary search (episode up to step hi)

The decisive mistake is assumed to lie between step {lo} and
 ↳ step {hi}. Is it in the first half (steps {lo} to {mid}) or
 ↳ in the second half (steps {mid + 1} to {hi})? Answer with
 ↳ JSON:
 {"half": "first" | "second", "component": "<the component most
 ↳ likely at fault>",
 "reason": "<one sentence>"}

Root-cause attributor

An LLM agent failed a task. Find the root cause of the failure.

Agent components (id: role):
 {COMP_TEXT}

Trace:
 {view(rec)}

Give (1) a one-sentence root cause describing the cause, not
 ↳ the symptom;
 (2) the index of the earliest step where the agent went wrong;
 (3) the single component whose change would most plausibly
 ↳ prevent this failure.
 Answer with JSON: {"root_cause": "...", "step": <int>,
 ↳ "component": "<component id>"}

Oracle (system prompt)

You construct ground truth for failure attribution. A
 ↳ customer-service agent failed a task of tau2-bench. You see
 ↳ the domain policy, the agent’s tools, the task’s grading
 ↳ criteria (the database must end as if exactly the gold
 ↳ actions were executed, and the agent must tell the user
 ↳ every required piece of information), and the conversation
 ↳ up to one step of the agent.

Decide whether the agent’s action at that step is a mistake: an
 ↳ action a careful agent following the policy would not take
 ↳ on the way to the gold outcome. Actions that are correct,
 ↳ or harmless and do not lead away from the gold outcome, are
 ↳ ok.

If it is a mistake, give the single action a careful agent would
 ↳ take at that step instead: one assistant turn, either a
 ↳ message to the user or one or more tool calls. Correct only
 ↳ this step; do not do more than a careful agent would do at
 ↳ this point. The corrected action must be one the agent
 ↳ could have taken with what it knew then (the conversation
 ↳ and tool results so far, and the policy); use the grading
 ↳ criteria only to know what the right outcome is, never to
 ↳ supply ids, amounts or facts the agent had not yet learned.

Answer with JSON:
 {"verdict": "ok" | "mistake", "why": "<one sentence>",
 "action": {"message": "<text to the user>" | {"tool_calls":
 ↳ [{"name": "<tool>", "arguments": {...}]}}}
 ("action" only when the verdict is mistake.)

Counterfactual search, suspect call (system prompt)

A customer-service agent on tau2-bench failed its task: the
 ↳ customer’s request was not resolved as the domain policy
 ↳ requires. You see the domain policy, the agent’s tools and
 ↳ the conversation; you do not know what the correct outcome
 ↳ was.

Name the steps where the agent most likely went wrong, most
 ↳ suspect first (at most 5). For each, give the single action
 ↳ a careful agent would have taken at that step instead (one
 ↳ assistant turn: a message to the user or one or more tool
 ↳ calls, using only what the agent knew at that point) and
 ↳ the harness component most likely at fault.

Harness components:

- prompt: the system prompt text (instructions and policy
→ wording the model reads)
- control_flow: the agent loop code: how the next action is
→ produced, checks or gates applied to tool calls before they
→ run, retries, enforced confirmations
- config: numeric settings such as temperature, token limits or
→ step limits
- output_plumbing: how tool results and tool descriptions are
→ rendered to the model
- context_mgmt: which parts of the conversation history the
→ model sees
- client_tool: extra tools the harness gives the model (e.g. a
→ checker or calculator)
- skill: instruction files loaded for specific situations
- memory: structured state the harness keeps across steps
- subagent: an extra model call, e.g. a second model reviewing
→ an action

Answer with JSON:
{"suspects": [{"step": <int>, "why": "<one sentence>",
→ "component": "<component>",
"action": {"message": "<text>" | {"tool_calls": [{"name":
→ "<tool>", "arguments": {...}]}}]}

Variants (post hoc). The reruns of the all-at-once judges and binary search in §5 use the same prompts with one sentence of the introduction replaced. Registered:

The DECISIVE MISTAKE is the earliest agent step such that, had
→ the agent acted correctly at that step and continued
→ normally, the task would most likely have succeeded.

Most likely rescue:

The DECISIVE MISTAKE is the agent step whose correction would
→ most likely have rescued the task: had the agent acted
→ correctly at that step and continued normally, the task
→ would most likely have succeeded. If several steps qualify,
→ choose the one where a correction is most likely to make
→ the task succeed, which need not be the earliest.

Largest gain over null:

The DECISIVE MISTAKE is the agent step where a correction would
→ make the biggest difference: the step at which acting
→ correctly, and then continuing normally, would most
→ increase the chance that the task succeeds compared with
→ letting the agent act again from that step without any
→ correction. A step from which the task would often have
→ succeeded anyway, without a correction, is a weak choice
→ even if a correction there would succeed.

The no-grading variant keeps the registered definition, removes the grading report and the customer's hidden instructions from the episode, gives the domain policy and tools (AppWorld: the agent's rules and the APIs) as counterfactual search receives them, and opens with: "... You see the domain policy and the agent's tools, but not the customer's instructions or the grading, so you do not know what the correct outcome was."

Repair experiment: notes to the proposer. RRSI's proposer prompt is unchanged. In the no-attribution group the round's analysis report is replaced by the first note below; the step groups get the second note and one digest per failing trace, {"task_id", "lens": "failure", "decisive_step": k , "step_excerpt": <the step, at most 600 characters>, "blocker": "failure decided at step k "}.
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898

τ^2	steps/dec.	utterance	read	write	transfer
Retail	1165/65	54%/28	34%/1	11%/36	1%/0
Airline	1026/37	55%/25	31%/4	12%/8	1%/0
AppWorld	steps/dec.	docs	read	write	submit
AppWorld	1961/55	27%/3	45%/9	14%/29	5%/9

Table 3: Share of all agent steps of each kind / number of best rescue steps of that kind. AppWorld "docs" looks up API documentation; 5 AppWorld best rescue steps (9% of steps) are other code.

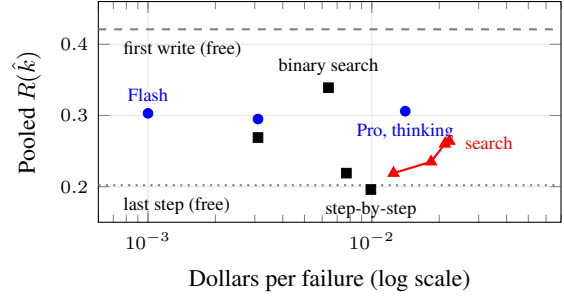


Figure 3: Rescue score against cost. Circles: all-at-once judges; squares: other LLM methods; triangles: counterfactual search at budgets 8, 16, 24 and 40. The free first-write rule lies above every method.

No analysis report this round. Read the traces yourself
→ (list_traces, then read_trace) to find what went wrong; the
→ failing traces are listed first.

No analysis report this round. Read the traces yourself
→ (list_traces, then read_trace) to find what went wrong; the
→ failing traces are listed first. For each failing trace,
→ PER-TASK DIGESTS give decisive_step: the step at which an
→ automatic failure-attribution method located the failure.
→ Read the trace around that step (read_trace with
→ from_step/to_step) to see what went wrong there.

C Secondary Metrics, Step Kinds and Search Budgets

Table 4 gives the secondary metrics, Table 3 the kinds of agent steps and best rescue steps, and Table 5 counterfactual search by replay budget. Search's suspects included a step with a gain of at least 0.5 on 122 of 157 rescuable failures (78%); its confirmations were the weak point. Of the 87 rescuable failures where its own test (4 corrected against 4 null replays) confirmed a suspect, the confirmed step had a gain of at least 0.5 in 41; elsewhere its corrections at the right step failed, or a weaker step passed the test first.

D Post-hoc Variants and Who&When

Robustness details. Choosing rescuable failures and scoring on the same noisy gains flattered the best rescue step (0.75, against 0.48 out of sample) but no method over another. With the observed

Method	Model	Exact [95%]	Within 1	Earliest	Share of max	$R(\hat{k})$, all failures
First write	rule	0.38 [0.27, 0.49]	0.62	0.33	0.52	0.23 [0.17, 0.30]
Last step	rule	0.13 [0.07, 0.20]	0.43	0.08	0.25	0.11 [0.07, 0.16]
All-at-once	Flash	0.28 [0.20, 0.36]	0.52	0.31	0.40	0.19 [0.14, 0.23]
All-at-once	Pro	0.31 [0.22, 0.39]	0.52	0.32	0.39	0.17 [0.13, 0.22]
All-at-once	Pro, thinking	0.32 [0.22, 0.42]	0.60	0.36	0.42	0.18 [0.14, 0.23]
Step-by-step	Pro	0.19 [0.13, 0.26]	0.38	0.22	0.25	0.11 [0.07, 0.15]
Binary search	Pro	0.32 [0.24, 0.40]	0.58	0.34	0.43	0.19 [0.15, 0.24]
Root-cause attributor	Pro	0.28 [0.19, 0.37]	0.49	0.31	0.36	0.16 [0.12, 0.20]
RRSI trace digester	Pro	0.21 [0.14, 0.29]	0.41	0.22	0.29	0.14 [0.10, 0.17]
Counterfactual search@40	Pro, replays	0.26 [0.18, 0.35]	0.44	0.31	0.35	0.15 [0.11, 0.19]

Table 4: Secondary metrics, pooled over the three domains. Exact, within one step, earliest rescuing step and share of the largest gain are over rescuable failures; the last column is over all 300 failures.

Budget	Retail	Airline	AppWorld	Pooled	\$
8	0.22	0.13	0.28	0.22	0.0125
16	0.21	0.23	0.27	0.23	0.0184
24	0.22	0.31	0.27	0.26	0.0212
32	0.22	0.31	0.28	0.26	0.0222
40	0.22	0.31	0.28	0.26	0.0222

Table 5: Counterfactual search: mean $R(\hat{k})$ on rescuable failures by replay budget, read off the same runs, and dollars per failure.

Reference	Resc.	First wr.	Best LLM	Rank	τ
Registered	157	0.42	Binary search 0.34	1	1.00
$K = 2$	188	0.37	Binary search 0.30	1	0.89
$K = 1$	181	0.39	Binary search 0.33	1	0.82
$\theta = 0.25$	226	0.31	Binary search 0.26	1	0.91
$\theta = 0.375$	177	0.38	Binary search 0.31	1	0.96
$\theta = 0.625$	116	0.50	Binary search 0.38	1	0.78
$\theta = 0.75$	107	0.54	Binary search 0.39	1	0.82
One rescuing step	102	0.33	Binary search 0.27	1	1.00
No null replays	203	0.37	Judge, thinking 0.42	6	0.60
Split samples	193	0.34	Binary search 0.27	1	0.96
Observed action as control	157	0.44	Judge, thinking 0.40	1	0.82
Second oracle (Flash)	120	0.38	Binary search 0.29	1	0.78
Both oracles averaged	121	0.43	Binary search 0.32	1	0.82
Flagged corrections dropped	152	0.43	Binary search 0.34	1	1.00

Table 6: Method scores under other reference choices (post hoc), pooled over the three domains: rescuable failures, the first write’s mean $R(\hat{k})$, the best LLM method and its $R(\hat{k})$ (judge, thinking: all-at-once with Pro and thinking), the first write’s rank among the ten methods of Table 1, and Kendall’s τ of the method ordering against the registered one ($K = 4$ oracle samples, null replays, threshold $\theta = 0.5$). Rows with fewer samples average over all subsets of the four; split samples choose rescuable failures on two samples and two null replays and score on the others; the observed action forced at step k replaces null replays on the registered rescuable failures; the second oracle is V4-Flash, and the last row averages the two oracles’ profiles.

action forced as the control, 156 of 157 failures stayed rescuable and the all-at-once judge with Pro rose from 0.30 to 0.39, leaving the first write ahead of the best registered judge by 0.04 $[-0.07, +0.15]$. The second oracle proposed corrections less often (0.16–0.26 of samples against 0.22–0.37), and the two oracles agreed on rescuability for 69–80% of failures ($\kappa = .36-.61$). Under its profile the first write led binary search by +0.08 $[+0.01, +0.16]$, and under the average of both profiles by +0.10 $[+0.02, +0.19]$ ($\tau = .82$); binary search asked for the largest gain came within 0.02 of it under both. The admissibility screen also flagged a number not seen before in 6.1% of corrections, mostly sums or differences the agent could compute; corrections of retail writes, the most effective kind, were flagged for an unseen value in none and for any unseen value in 0.4%. Without the corrections flagged for an unseen value or two more calls, 152 failures were rescuable, the first write scored 0.43 and binary search 0.34 (+0.09 $[-0.01, +0.19]$), and binary search asked for the largest gain came within 0.02 (+0.02 $[-0.07, +0.10]$). Without null replays a step scores $R_k + (m_k/K) \hat{p}_k^{\text{null}}$, where m_k of the K samples corrected it; at the registered rescuable failures the added term averaged 0.08 to 0.13 for the LLM methods and 0.04 for the first write.

Table 7 compares the judges’ registered prompts with the re-prompted variants (Appendix B), Table 8 the gold-deviation rule with other methods on τ^2 , Tables 9 and 10 split rescue gains into how often the oracle proposed a correction and how well its corrections worked, and Table 11 screens the corrections for information the agent had not seen and for scope. A stratified sample of 60 corrections (20 per domain, half of them flagged, spread over kinds of step) is released with an audit sheet asking whether each uses unavailable information, does more than one careful step, and is a step a careful agent could take.

Method	Definition	Retail	Airline	AppWorld	Pooled [95%]	Exact	Position	\$	Difference [95%]
All-at-once, Flash	earliest (reg.)	0.29	0.41	0.25	0.30 [0.24, 0.37]	0.28	0.64	0.0010	
	most likely rescue	0.36	0.35	0.26	0.32 [0.25, 0.39]	0.32	0.68	0.0010	+0.02 [−0.04, +0.08]
	largest gain over null	0.29	0.41	0.28	0.31 [0.24, 0.39]	0.34	0.64	0.0010	+0.01 [−0.04, +0.06]
All-at-once, Pro	earliest (reg.)	0.30	0.34	0.26	0.30 [0.23, 0.36]	0.31	0.63	0.0031	
	most likely rescue	0.33	0.47	0.33	0.36 [0.29, 0.44]	0.38	0.65	0.0031	+0.07 [+0.01, +0.12]
	largest gain over null	0.39	0.42	0.29	0.36 [0.29, 0.44]	0.36	0.67	0.0031	+0.07 [+0.02, +0.12]
	earliest, no grading	0.17	0.32	0.25	0.23 [0.17, 0.29]	0.24	0.58	0.0028	−0.07 [−0.13, −0.00]
Binary search, Pro	earliest (reg.)	0.33	0.34	0.35	0.34 [0.27, 0.41]	0.32	0.69	0.0064	
	most likely rescue	0.38	0.42	0.32	0.37 [0.29, 0.44]	0.35	0.72	0.0069	+0.03 [−0.04, +0.10]
	largest gain over null	0.44	0.42	0.35	0.40 [0.33, 0.48]	0.39	0.74	0.0071	+0.07 [−0.01, +0.13]
	earliest, no grading	0.21	0.24	0.24	0.23 [0.16, 0.29]	0.20	0.66	0.0064	−0.11 [−0.20, −0.02]
Counterfactual search@0	first suspect, no grading	0.22	0.13	0.28	0.22 [0.16, 0.28]	0.20	0.52	0.0037	
Counterfactual search@40	no grading	0.22	0.31	0.28	0.26 [0.20, 0.33]	0.26	0.57	0.0222	+0.04 [+0.00, +0.09]
First write	rule	0.53	0.27	0.39	0.42 [0.33, 0.51]	0.38	0.78	0	

Table 7: Judges with their registered prompt (earliest decisive mistake) and post hoc variants: the step most likely to rescue the task, the step with the largest gain over letting the agent act again, and the registered definition without grading information or hidden instructions (what counterfactual search sees); and search with and without its replays. On the 157 rescuable failures: mean $R(\hat{k})$, exact agreement with the best rescue step, mean relative position of the named step, dollars per failure (mean over domains), and the paired difference in pooled $R(\hat{k})$ from the registered prompt (search@40: from search@0) with a cluster-bootstrap interval.

Method	Retail (65)	Airline (37)	τ^2 (102) [95%]	Exact	Gold deviation — method [95%]
Gold deviation (rule)	0.58	0.35	0.50 [0.38, 0.60]	0.49	
First write (rule)	0.53	0.27	0.44 [0.32, 0.55]	0.43	+0.06 [+0.01, +0.12]
All-at-once, Pro	0.30	0.34	0.32 [0.23, 0.40]	0.33	+0.18 [+0.02, +0.33]
Binary search, Pro	0.33	0.34	0.33 [0.24, 0.43]	0.34	+0.16 [+0.02, +0.30]
Counterfactual search@40	0.22	0.31	0.25 [0.17, 0.34]	0.25	+0.24 [+0.09, +0.38]

Table 8: The gold-deviation rule (post hoc): the first state-changing call that matches no unused gold action, else the last step (20% of failures). Mean $R(\hat{k})$ on the 102 rescuable τ^2 failures and paired differences with cluster-bootstrap intervals.

Method	Model	Proposed	Gain if proposed	$R(\hat{k})$
First write	rule	0.64	0.61	0.42
Last step	rule	0.53	0.33	0.20
All-at-once	Flash	0.58	0.51	0.30
All-at-once	Pro	0.60	0.48	0.30
All-at-once	Pro, thinking	0.61	0.49	0.31
Step-by-step	Pro	0.52	0.34	0.20
Binary search	Pro	0.64	0.49	0.34
Root-cause attributor	Pro	0.56	0.46	0.27
RRSI trace digester	Pro	0.57	0.34	0.22
Counterfactual search@40	Pro, replays	0.59	0.44	0.26
Best rescue step		0.88	0.87	0.76

Table 9: $R(\hat{k})$ split into the share of oracle samples that proposed an applicable correction at the named step and the mean gain of those corrections (over failures with at least one), on the 157 rescuable failures (post hoc).

Domain	Step	Steps	Proposed	Gain if proposed	Null
Retail	utterance	630	0.35	0.13	0.25
	read	400	0.24	0.02	0.45
	write	125	0.51	0.52	0.04
Airline	utterance	569	0.38	0.13	0.16
	read	321	0.33	0.08	0.31
	write	127	0.48	0.18	0.02
AppWorld	docs	535	0.07	0.20	0.13
	read	885	0.13	0.12	0.13
	write	272	0.53	0.21	0.13
	submit	91	0.65	0.32	0.01

Table 10: The same split over all steps of all 300 failures by kind of step, with the success of null replays where they were run (post hoc; τ^2 transfers and other AppWorld code omitted).

Who&When. To check that our judges are not unusually weak, we ran the all-at-once, step-by-step and binary-search judges on the 184 logs of Who&When with its authors’ prompts and the setting that shows the ground-truth answer (Zhang et al., 2025). They identified the responsible agent in 58–67% of logs with all-at-once judging and the responsible step in 3–25% (Table 12), the same

pattern the benchmark reports: agents are found far more often than steps.

E Repair Candidates

Figure 4 shows the held-out gain of every deployable candidate in the repair experiment.

Domain	Corrections	Unseen	Unseen	More calls		Calls
	scored	value	number	$\geq +2$	$\geq +1$	obs. / corr.
Retail	1,554	0.4	7.6	0.4	1.5	0.94 / 0.68
Airline	1,549	1.6	10.0	2.3	3.7	1.30 / 0.80
AppWorld	1,692	0.2	1.1	2.7	15.9	0.96 / 1.04
At best rescue steps	553	0.2	2.9	2.9	5.8	

Table 11: Mechanical screen of the oracle’s corrections (post hoc): every correction that entered a registered profile, in %. Unseen value: an argument (an identifier, name, e-mail or date; in AppWorld a value passed to an API to look something up) that appears neither in the episode before the step nor in the policy, tool schemas or API documentation. Unseen number: a number above 10 likewise absent, often a sum or difference the agent could compute. More calls: tool or API calls beyond $\max(1, \text{observed})$. Re-scored without the corrections flagged for an unseen value or two more calls, the ranking was unchanged (Table 6).

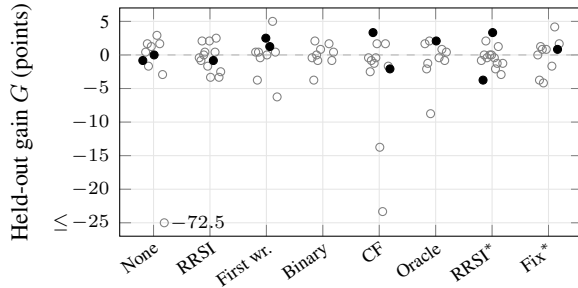


Figure 4: Held-out gain of every deployable candidate in the repair experiment, by the evidence its proposer got (*: no reading of failing traces; Fix: oracle step, diagnosis and correction). Filled: accepted by RRSI’s own rule. One candidate (-72.5) is drawn at -25 .

F Registration and Deviations

The registration (methods, prompts, metrics, the primary comparison and the budget) was committed before any method ran on the main set. Changes to the attribution study, each logged before the data it affected were analysed:

- **Reference protocol.** The registration left the protocol to be chosen on the pilot by test-retest agreement. After the earliest-step protocol agreed on 42% of pilot pairs, protocol 3 and the rescue-gain metrics replaced it (Addendum 1), while methods were running and before any main-set profile existed.
- **Retail top-up.** The first loop’s harnesses yielded 86 distinct retail failures under the sampling rule; 14 more were drawn with the same rule from two further runs of the loop, as registered. All methods moved by at most 0.02 when they were added.

- **AppWorld.** Registered in Addendum 2 before any AppWorld failure was drawn. Two corrected replays of one step failed on every attempt because the oracle’s code passed a literal ellipsis that AppWorld’s request log cannot serialise; they count as corrections not applied. That failure’s other corrections at the step rescued nothing and its largest gain was 0.375, so they could not have made it rescuable.

Repair experiment. Registered separately, before any attribution was run on its traces and before any candidate was drafted. As checks, the RRSI-analysis group’s mean gain (-0.34) matched that of the candidates the original rounds had drafted from the same evidence (-0.42), and re-deploying each incumbent moved its held-out success by -2.5 to $+3.3$ points against its earlier deployment, within the noise of 240 episodes. The first state served as a pilot; it exposed no bug, so its cells were kept, as registered. Two operational problems had no effect on data: a lookup by abbreviated commit hash re-invoked finished deployments, which skipped every episode as already done, and a container restart interrupted the deployments, which resumed with finished episodes kept. One secondary test changed. The registered code tested the round-level contrasts by permuting slots, but a round has one accepted candidate, so each cell’s value was copied to its two slots and the permutation split the copies between groups, which is not a valid test. We noticed this after the first run of the analysis, which gave $p = .009$ for first write against no attribution, and replaced the test by an exact sign-flip test over the eight state differences ($p \geq .25$); estimates and intervals are unchanged.

Post-hoc analyses. Not registered: the re-scoring under other reference choices, the null-replay rates by step position, the decomposition of the null-free score and the retest agreement among failures both runs found rescuable (Table 6, Figure 2; `attrib/robustness.py`); the split of $R(\hat{k})$ into proposals and their gains, search’s first suspect, the null replays’ reproduction of the observed action and the split-sample scores (Tables 9, 10; `attrib/review2.py`); the observed-action control (`attrib/orig_control.py`); the second oracle (`attrib/second_oracle.py`, `attrib/groundtruth3.py` with another oracle model, sharing null replays where both oracles flagged a step); the judges with other defini-

Method	Model	Hand-crafted (58)		Generated (126)	
		agent	step	agent	step
All-at-once	Flash	0.59	0.09	0.58	0.21
All-at-once	Pro	0.62	0.03	0.67	0.18
All-at-once	Pro, thinking	0.62	0.15	0.65	0.25
Step-by-step	Pro	0.24	0.19	0.44	0.32
Binary search	Pro	0.26	0.09	0.38	0.04

Table 12: Who&When, with ground truth shown to the judge as in its authors’ setting: accuracy of the responsible agent and step.

tions or without grading information (Table 7; attrib/rescue_prompt.py); the gold-deviation rule (Table 8; attrib/gold_rule.py); the admissibility screen and the audit sample (Table 11; attrib/admissibility.py); counterfactual search with grading information and the factorial analysis (attrib/search.py -informed, attrib/factorial.py); in the repair experiment, the rescue score of each group’s pointers on its own failures (attrib/phaseb_accuracy.py), the oracle-pointer group, run after the registered analysis with held-out results unseen until its deployments had finished (attrib/phaseb_oracle.py), the proposers’ reading of traces (attrib/phaseb_reading.py), the no-attribution group’s mean without its worst candidate, the intervals with states grouped by run and the correlation of pointer scores with gains (attrib/phaseb_review3.py), and the two no-reading groups, run with RRSI’s proposer blocked from failing traces and held-out results unseen until their deployments had finished (attrib/phaseb.py, attrib/phaseb_review3.py -noread).